

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1710

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 20%

Duration : 1 Hour

QUESTIONS: 5 Total Marks: 50

Annexure A

Reverse Engineering

Code of Conduct:

I Kanish Kumar S (192511415) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kanish Kumar S

Student 12: Kanish Kumar S | Register Number: 192511415

Reverse Engineering a Video Streaming Recommendation Engine

A video streaming platform recommends the next video to millions of users based on watch history and real-time engagement signals. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how ranking decisions are made under strict latency constraints.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Answer: Reverse Engineering a Video Streaming Recommendation Engine

1. Problem Restatement and Reverse-Engineering Approach

The platform must serve a ranked "next video" recommendation to millions of concurrently active users, using two categories of signal: (a) long-term watch history, which is large in volume but changes slowly, and (b) real-time engagement events (play, pause, seek, skip, like), which are small in size but must be reflected in the ranking within milliseconds. Reverse engineering such a system means working backward from its observable behaviour — instant re-ranking after a skip, session-aware suggestions, and sub-second response times — to infer the most plausible internal design. Three sub-systems must be reconstructed: (i) a fault-tolerant, RDD-based streaming pipeline that ingests and transforms engagement data, (ii) a set of cooperating intelligent agents that convert raw signals into a personalised candidate set, and (iii) a latency-bounded ranking mechanism that converts candidates into an ordered list before the response deadline expires.

2. RDD-Based Streaming Data Pipeline

At the core of the ingestion layer sits Spark's Resilient Distributed Dataset (RDD) abstraction, operating through the micro-batch model of Structured/DStream streaming. Each short time slice (e.g., 100–500 ms) of incoming Kafka events is materialised as an immutable, partitioned RDD, which gives the pipeline three properties essential at this scale: automatic recovery via lineage (a failed partition is recomputed from its parent RDDs rather than requiring a checkpoint restore), horizontal scalability (partitions are processed in parallel across the cluster, keyed by `user_id` so a user's events stay co-located), and composability (each stage is a pure transformation, so stages can be reordered, tested, and reused).

2.1 Pipeline Stages

- **Ingestion:** client and server events (clicks, pauses, seeks, completions) are published to partitioned Kafka topics; a DStream receiver converts each micro-batch interval into a new RDD.
- **Cleaning / parsing:** `map()` and `flatMap()` deserialize raw JSON/Avro events into typed case classes; `filter()` discards malformed or bot-generated events.
- **Session aggregation:** `reduceByKeyAndWindow()` combines events by `user_id` over a sliding window (e.g., last 5 minutes) to produce session-level features such as recent genres watched, average watch completion, and skip rate — without re-scanning the entire event history.
- **Feature enrichment:** the aggregated RDD is `join()`-ed with a broadcast/reference RDD of content metadata (genre, cast, tags, release year), producing a feature RDD ready for scoring.
- **Persistence:** `persist()/cache()` keeps hot RDDs in memory across the DAG to avoid recomputation, and results are written to a low-latency key-value feature store (e.g., Redis) that agents can read with single-digit-millisecond latency — RDD checkpointing to distributed storage covers long-running state and failure recovery.

This pipeline reconciles the two timescales in the problem: the streaming RDD path handles the millisecond-level engagement signals, while a parallel batch RDD/DataFrame job periodically recomputes long-term watch-history embeddings and writes them to the same feature store, so both freshness and depth of history are available to downstream ranking.

3. Agent Interactions in the Recommendation System

Because a single monolithic model cannot simultaneously maintain per-user state, retrieve from a catalog of millions of videos, and respect a strict time budget, the system is best modelled as a small society of cooperating agents, coordinated through a publish–subscribe message bus rather than direct function calls. This keeps agents loosely coupled and lets any one of them be scaled, replaced, or bypassed independently.

3.1 Agent Roles

- **User-Profile Agent:** reads the feature store and maintains a continuously updated "taste vector" per user, blending long-term embeddings with the current session's real-time signals.
- **Candidate-Generation Agent:** performs approximate nearest-neighbour (ANN) retrieval over the video-embedding index, using the taste vector to shortlist on the order of a few thousand plausible candidates from a catalog of millions — this step trades ranking precision for speed.
- **Context Agent:** filters or re-weights candidates using situational context — device type, time of day, network bandwidth (e.g., avoiding 4K-only suggestions on a poor connection), and content availability by region.
- **Diversity / Exploration Agent:** guards against feedback-loop staleness by injecting novel or exploratory items using a multi-armed-bandit style strategy, preventing the recommendation list from over-fitting to past clicks.
- **Orchestrator / Coordinator Agent:** sequences the pipeline of agent calls, enforces the overall latency budget, and short-circuits (returns a cached or default list) if any agent overruns its allotted time slice.

The interaction pattern is largely a pipeline with feedback: each agent narrows or re-weights the candidate set it receives from the previous agent, while the User-Profile Agent is itself continuously updated by the streaming layer, so the effect of a skip or a completed episode can influence the very next request. This is what reverse engineering the platform's visible responsiveness implies about its internal structure — recommendations change immediately because agent state, not just model weights, is updated in real time.

4. Ranking Decisions Under Strict Latency Constraints

A user-facing "next video" call typically must resolve in well under 200 milliseconds end to end, which rules out running an expensive deep model over every candidate. The system therefore reverse engineers naturally into a funnel, or cascade, architecture that spends increasingly expensive computation on increasingly fewer items.

4.1 Two-Stage (Funnel) Ranking

- **Stage 1 — Lightweight scoring:** a simple, highly parallelisable model (linear model, shallow tree, or dot-product similarity) scores every one of the thousands of candidates from the Candidate-Generation Agent almost instantly, and only the top-K (commonly tens of items) are retained.
- **Stage 2 — Deep ranking:** a heavier model (deep neural network or gradient-boosted ensemble with cross-features) evaluates only that small top-K set, often in a single batched GPU call, producing the final order.

4.2 Additional Latency-Control Mechanisms

- **Latency budget / timeout guard:** the Orchestrator Agent attaches a deadline to the request; if candidate generation or ranking exceeds its slice of the budget, the system falls back to a cached or previously computed list rather than blocking the user.
- **Precomputation and caching:** user embeddings, candidate shortlists, and even entire recommendation lists are precomputed ahead of the request wherever behaviour is predictable, converting an online computation into a fast cache lookup.
- **Feature-store reads instead of recomputation:** ranking models read already-aggregated features from the low-latency store built in the RDD pipeline rather than recomputing aggregates per request.
- **Model and infrastructure optimisation:** techniques such as model quantisation, batching concurrent requests to the GPU, and colocating serving infrastructure close to the feature store reduce constant-factor latency at the serving layer.

The net effect is that ranking quality is deliberately traded off against response time in a staged manner: cheap, approximate decisions prune the search space early, and precise, expensive decisions are reserved for a small enough set that they fit inside the remaining time budget.

5. Consolidated System Architecture

The diagram below consolidates the reverse-engineered pipeline: event sources feed a Kafka-buffered ingestion layer; an RDD-based streaming layer performs micro-batch transformation, windowed aggregation, and feature enrichment before writing to a low-latency feature store; a chain of cooperating agents converts stored features into a ranked candidate set; and a two-stage, budget-guarded ranking layer produces the final response, with real-time feedback events looping back into the ingestion layer to keep the User-Profile Agent continuously current.

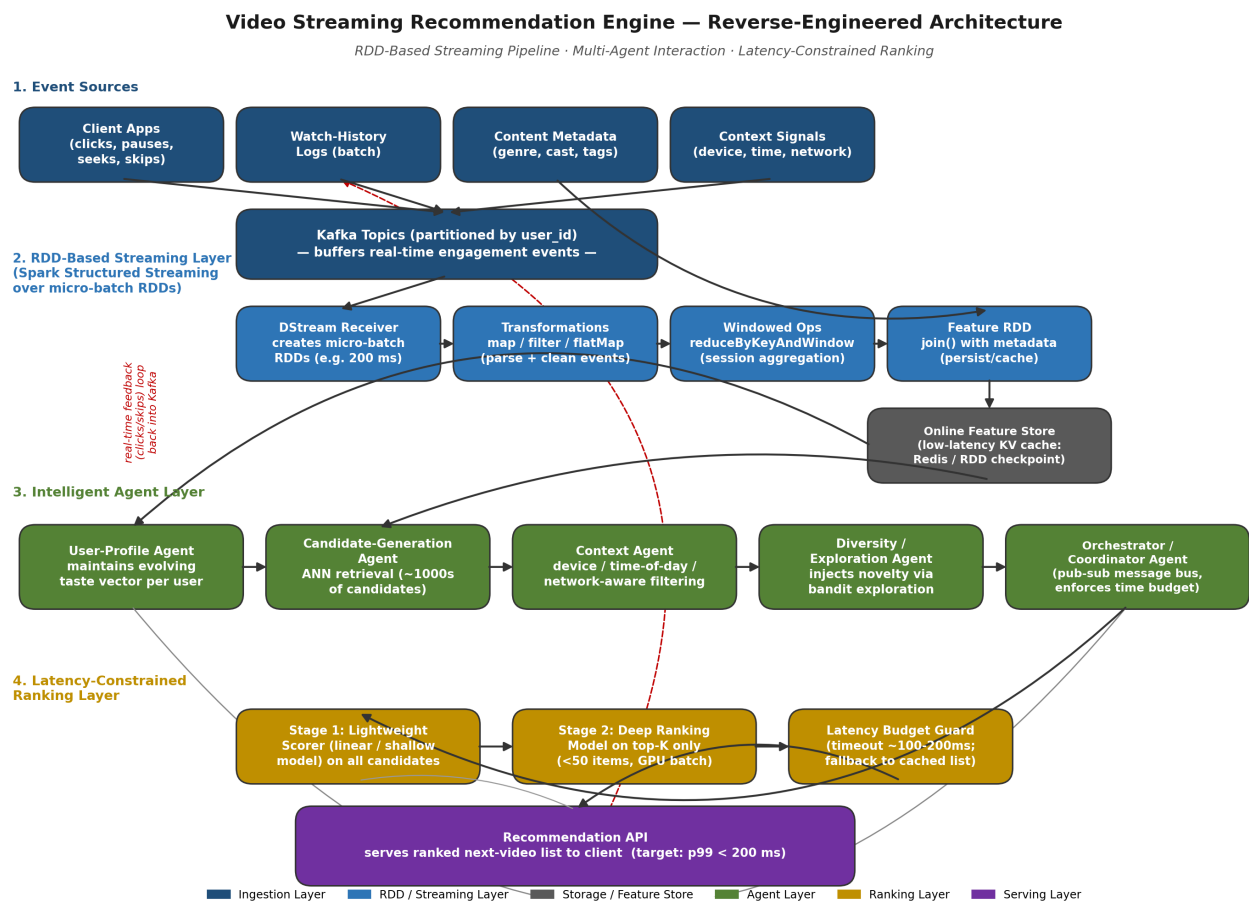


Figure 1: Reverse-engineered architecture of the video streaming recommendation engine — RDD-based streaming, multi-agent interaction, and latency-constrained ranking.

6. Summary

Reverse engineering the platform from its observable behaviour points to a system built on three coordinated layers. The RDD-based streaming layer supplies fault-tolerant, low-latency ingestion and feature aggregation from real-time engagement events. A society of specialised agents — profile, candidate generation, context, diversity, and orchestration — cooperates through a message bus to turn those features into a personalised, context-aware candidate set. Finally, a latency-constrained, two-stage ranking mechanism, backed by caching and strict timeout guards, converts that candidate set into an ordered recommendation within the platform's strict response-time budget. Together these layers explain how the system can react to a single skip or click within moments while still ranking from a catalog of millions of videos for a user base of millions.